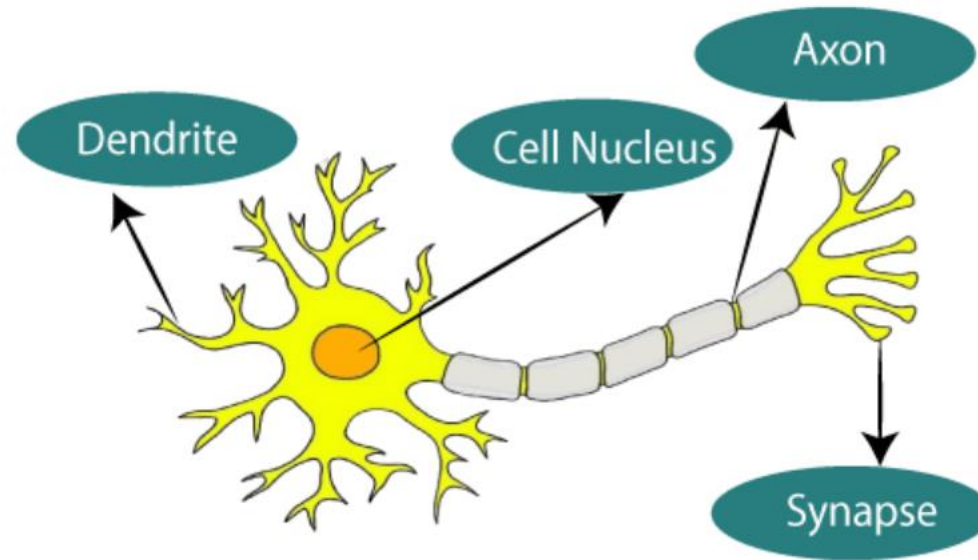


MODULE-1

NEURAL NETWORKS

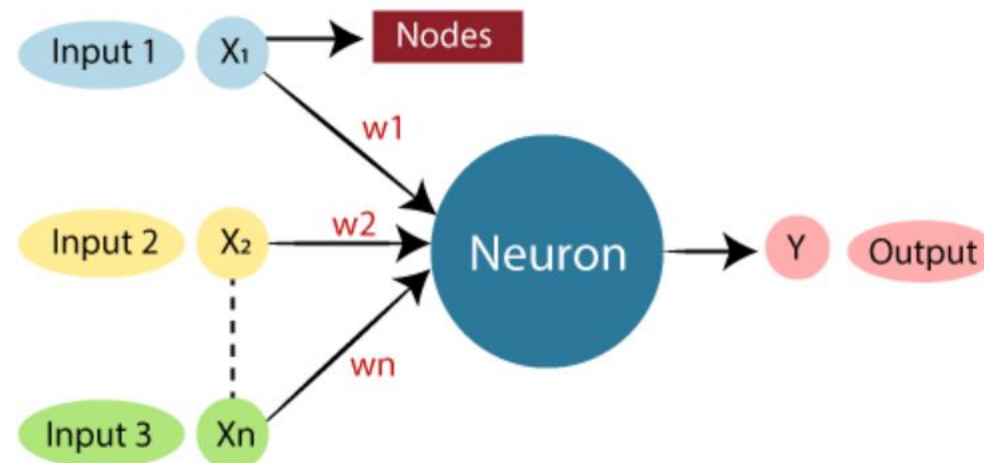
Neural Network

- The term "**Artificial Neural Network**" or "**Neural Networks**" or is derived from Biological neural networks that develop the structure of a human brain.
- Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks.
- These neurons are known as nodes.
- An **Artificial Neural Network** in the field of **Artificial intelligence** where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner



The given figure illustrates the typical diagram of Biological Neural Network.

The typical Artificial Neural Network looks something like the given figure.



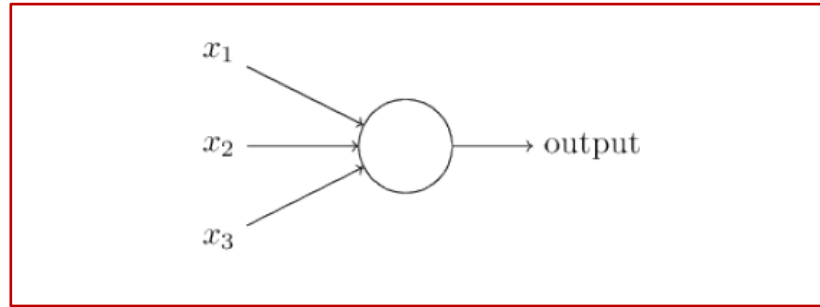
Neural network is a set of neurons organized in layers. Each neuron is a mathematical operation that takes its input, multiplies it by its weights and then passes the sum through the activation function to the other neurons. Neural network is learning how to classify an input through adjusting its weights based on previous examples.

What is an artificial neuron?

An artificial neuron is a mathematical function based on a model of biological neurons, where each neuron takes inputs, weighs them separately, sums them up and passes this sum through a nonlinear function to produce output.

Perceptron: Building Block of Artificial Neural Network

- Perceptrons were developed in the 1950s and 1960s by the scientist Frank Rosenblatt,
- perceptron takes several binary inputs, $x_1, x_2, \dots$, and produces a single binary output



- Rosenblatt proposed a simple rule to compute the output. He introduced *weights*, $w_1, w_2, \dots$, real numbers expressing the importance of the respective inputs to the output.
- The neuron's output, 0 or 1, is determined by whether the weighted sum $\sum_j w_j x_j$ is less than or greater than some *threshold value*.
- Just like the weights, the threshold is a real number which is a parameter of the neuron.

$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

How does it work?

- The perceptron works on these simple steps
 1. All the inputs x are multiplied with their weights w
 2. *Add* all the multiplied values and call them *Weighted Sum*.
 3. *Apply* that weighted sum to the correct *Activation Function*. For Example: Unit Step Activation Function.

The original **Perceptron** was designed to take a number of **binary** inputs, and produce one **binary** output (0 or 1).

The idea was to use different **weights** to represent the importance of each **input**, and that the sum of the values should be greater than a **threshold** value before making a decision like **true** or **false** (0 or 1).

✓ **Why do we need Weights and Bias?**

Weights shows the strength of the particular node.

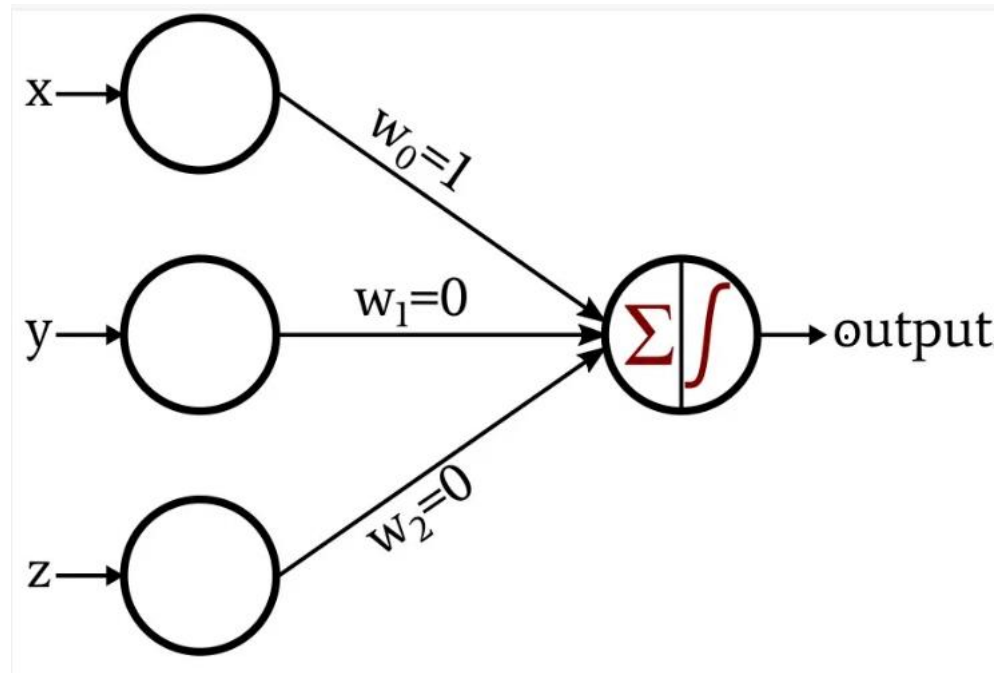
A *bias* value allows you to shift the activation function curve up or down.

✓ **Why do we need Activation Function?**

In short, the activation functions are used to map the input between the required values like (0, 1) or (-1, 1).

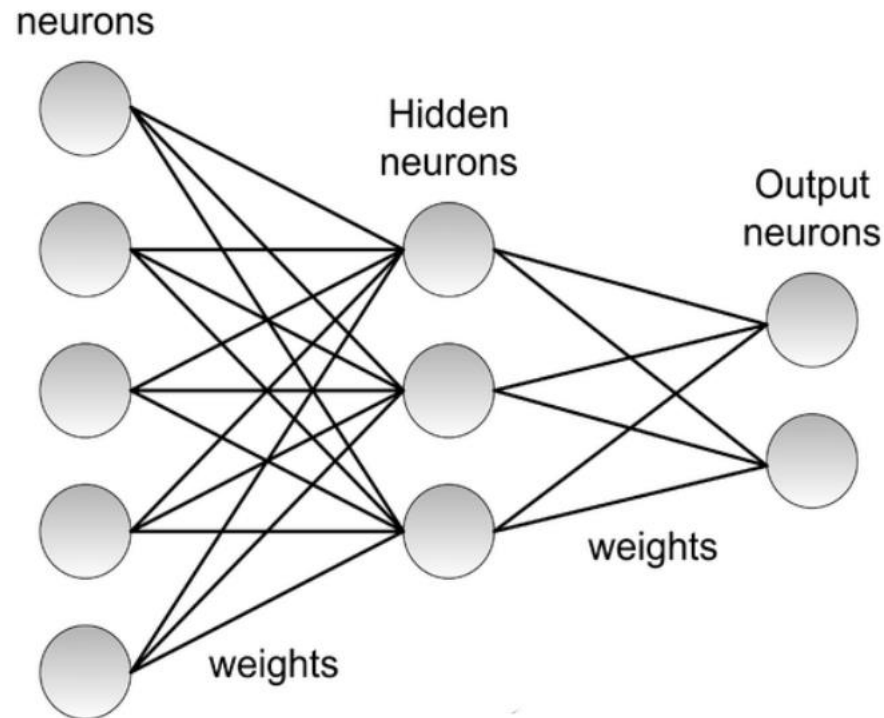
Single layer perceptrons

- A single-layer perceptron is the basic unit of a neural network. A perceptron consists of input values, weights and a bias, a weighted sum and activation function.



Multi Layer Perceptrons(MLPs)

- A Multilayer Perceptron has input and output layers, and one or more **hidden layers** with many neurons stacked together.



Activation Functions

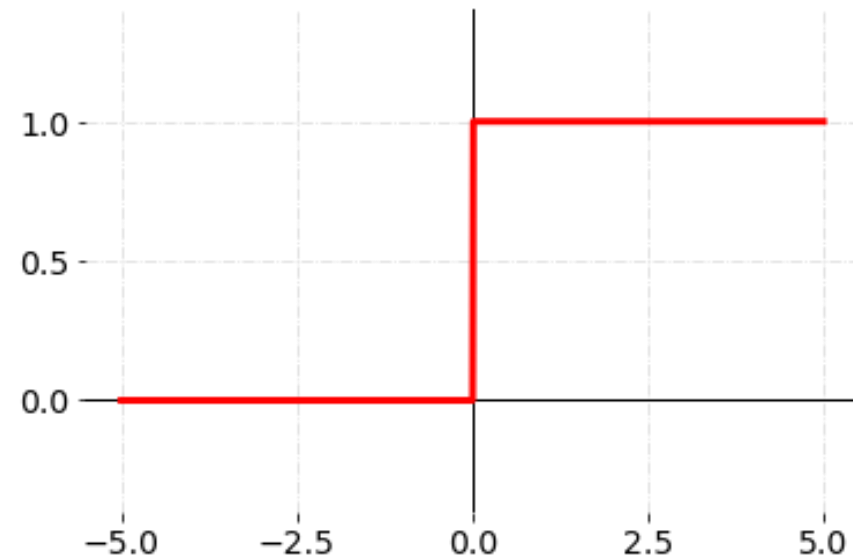
- Activation functions refer to the functions used in neural networks to compute the weighted sum of input and biases, which is used to choose the neuron that can be fire or not.
- **An Activation Function** decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction using simpler mathematical operations.
- The role of the Activation Function is to derive output from a set of input values fed to a node (or a layer).

Types of Activation Function

1. Binary step function

- Binary step function is a threshold-based activation function which means after a certain threshold neuron is activated and below the said threshold neuron is deactivated.
- This activation function can be used in binary classifications as the name suggests, however it can not be used in a situation where you have multiple classes to deal with.

$$\begin{aligned} f(x) &= 1, \text{ if } x \geq 0 \\ f(x) &= 0, \text{ if } x < 0 \end{aligned}$$

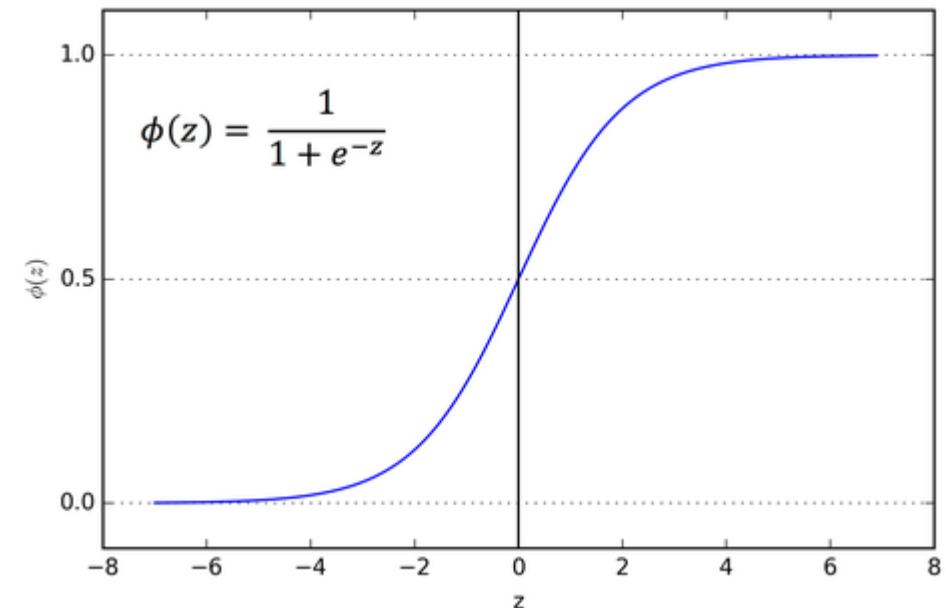


Types of Activation Function

2. Sigmoid or Logistic Activation Function

- The Sigmoid Function curve looks like a S-shape.
- The main reason why we use sigmoid function is because it exists between **(0 to 1)**.
- Therefore, it is especially used for models where we have to **predict the probability** as an output.
- Since probability of anything exists only between the range of **0 and 1**, sigmoid is the right choice.

$$f(x) = \frac{1}{1 + e^{-x}}$$

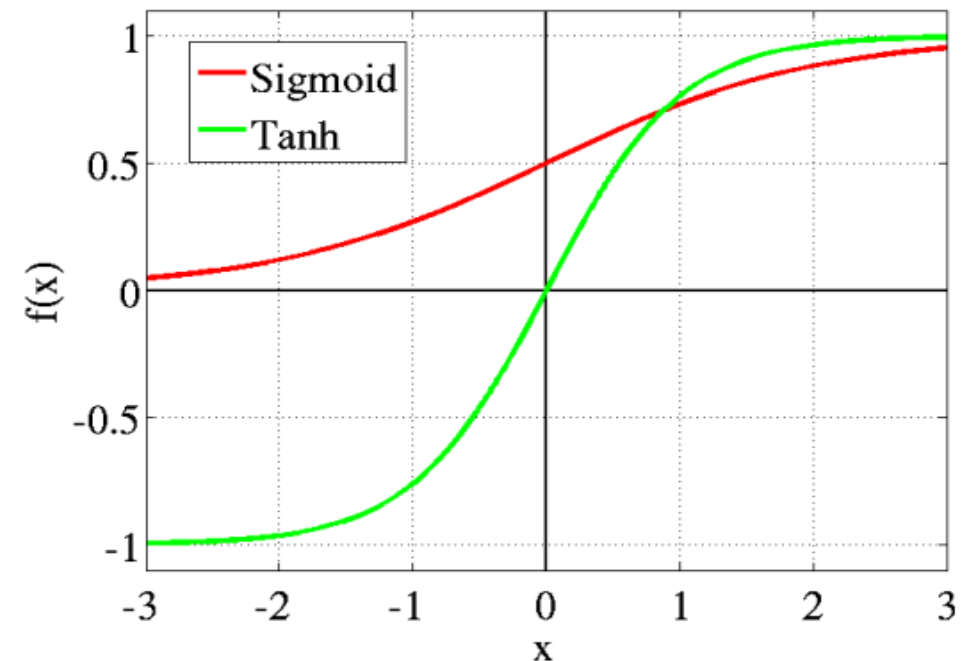


3. Tanh or hyperbolic tangent Activation Function

- tanh is also like logistic sigmoid but better.
- The range of the tanh function is from (-1 to 1). tanh is also sigmoidal (s-shaped).
- The tanh function is mainly used classification between two classes.

$$\begin{array}{c} \text{Tanh} \\ f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})} \end{array}$$

Both tanh and logistic sigmoid activation functions are used in feed-forward nets

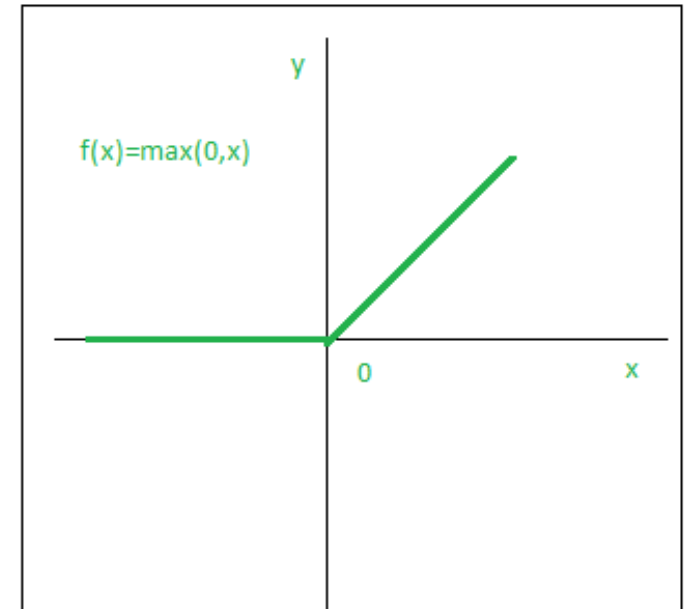


4. Rectified linear unit(ReLU)

- The ReLU function is the Rectified linear unit. It is the most widely used activation function. It is defined as:

$$f(x) = \max(0, x)$$

- The main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time.
- That is, if the input is negative it will convert it to zero and the neuron does not get activated.



5. Softmax activation function

- It is popularly used for multiclass classification problems.
- The input values can be positive, negative, zero, or greater than one, but the softmax transforms them into values between 0 and 1 (and they sum to 1), so that they can be interpreted as [probabilities](#).

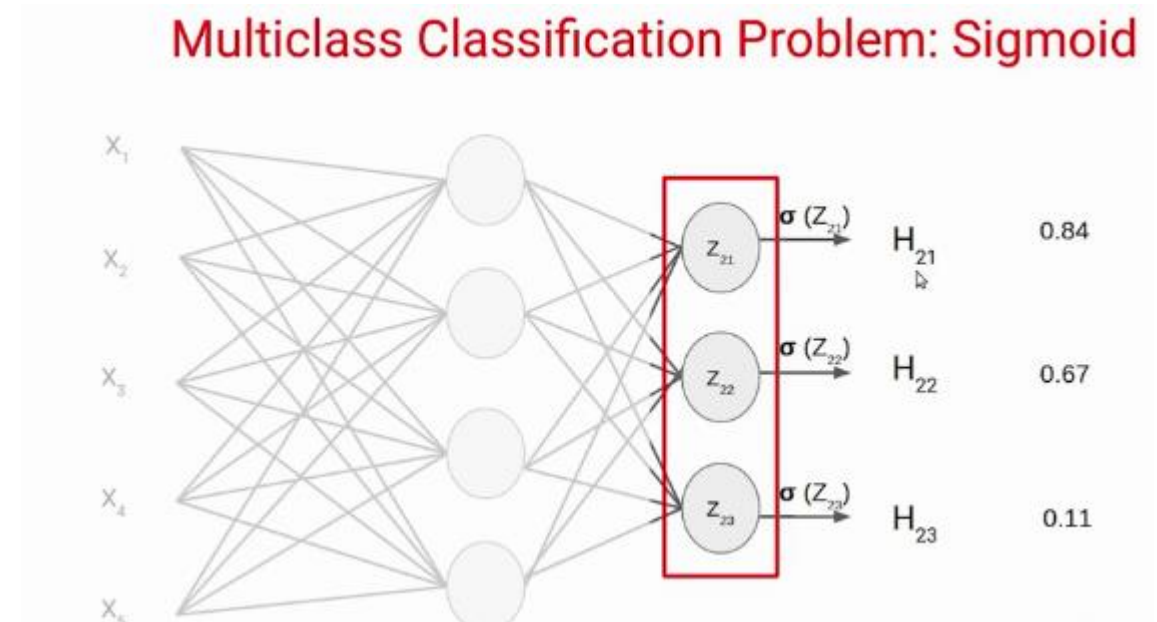
$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

- The Softmax activation function calculates the relative probabilities

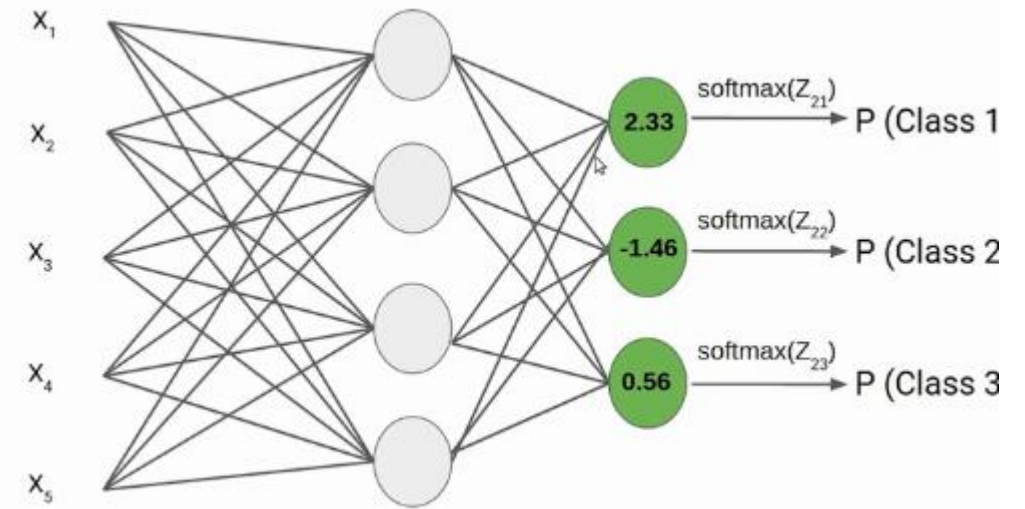
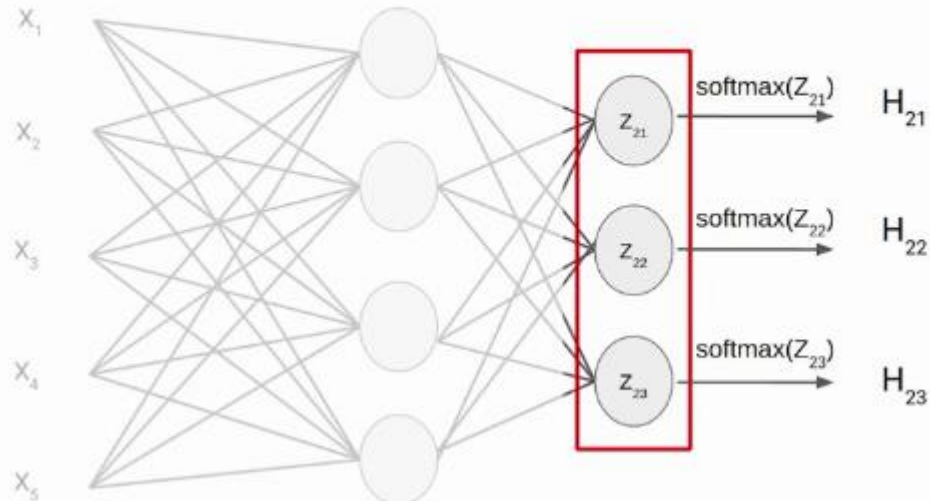
Why Softmax, and not Sigmoid for multiclass classification?

There are two problems in this case-

- First, if we apply a threshold of say 0.5, this network says the input data point belongs to two classes.
- Secondly, these probability values are independent of each other.
- That means the probability that the data point belongs to class 1 does not take into account the probability of the other two classes.
- This is the reason the sigmoid activation function is not preferred in multi-class classification problems.



Multiclass Classification Problem: Softmax



Example :

$$\begin{matrix} \text{2.33} \end{matrix} \rightarrow P(\text{Class 1}) = \frac{\exp(2.33)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)} = 0.83827314$$

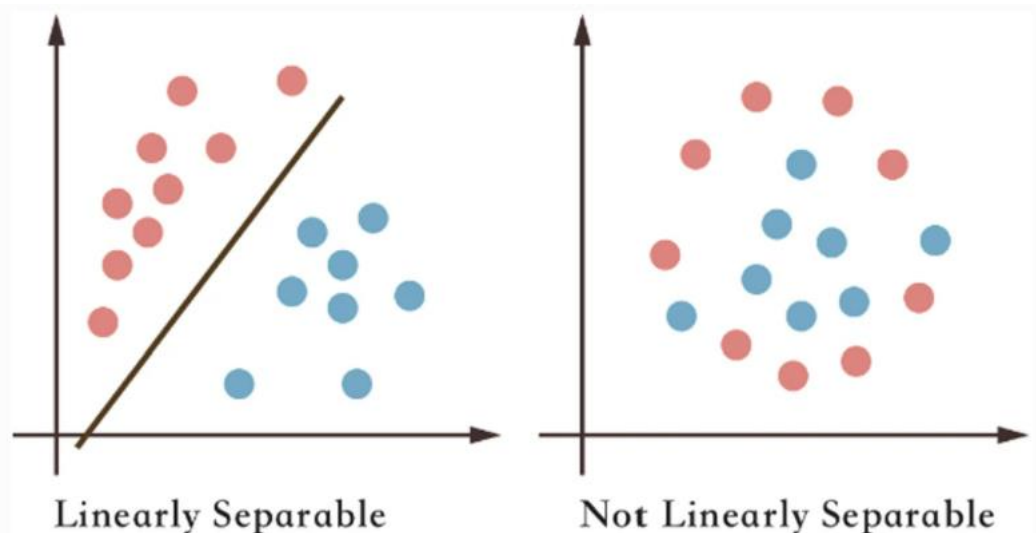
$$\begin{matrix} -1.46 \end{matrix} \rightarrow P(\text{Class 2}) = \frac{\exp(-1.46)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)} = 0.01894129$$

$$\begin{matrix} 0.56 \end{matrix} \rightarrow P(\text{Class 3}) = \frac{\exp(0.56)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)} = 0.14278557$$

In this case it clear that the input belongs to class 1. So if the probability of any of these classes is changed, the probability value of the first class would also change.

Linearly Separable

- Linear separability is the concept wherein the separation of input space into regions is based on whether the network response is positive or negative
- Two sets of points (or classes) are called linearly separable, if at least one straight line (decision line) in the plane exists so that **all the points of one class are on one side of the line and all the points of the other class are on the other side** they are called linearly separable



If there exists weights (with bias) for which training input vectors have positive (correct) response, lie on one side of the decision boundary and negative (incorrect) response, lie on the other side of the decision boundary, then the problem is “linearly separable.”

Representational power of perceptron

- A single perceptron can be used to represent many boolean functions.
- Perceptron can be used to represent functions which are linearly separable.
- For example, if we assume boolean values of 1 (true) and -1 (false), then one way to use a two-input perceptron to implement the AND function is to set the weights $w_0 = -3$, and $w_1 = w_2 = .5$.
- This perceptron can be made to represent the OR function instead by altering the threshold to $w_0 = -.3$.
- Perceptron can represent all of the primitive boolean functions AND, OR, NAND (1 AND), and NOR (1 OR). (linearly separable)
- Unfortunately, however, some boolean functions cannot be represented by a single perceptron, such as the XOR function .

XOR problem

- XOR is not linearly separable
- XOR cannot be represented by a single layer perceptron, but by MLP

LOSS FUNCTIONS

- Machines learn by means of a loss function.
- It's a method of evaluating how well specific algorithm models the given data.
- If predictions deviates too much from actual results, loss function would give a very large number
- Gradually, with the help of some optimization function, loss function learns to reduce the error in prediction

REMEMBER:

The objective is to minimize the loss between the predictions and the actual outputs

CLASSIFICATION OF LOSS FUNCTIONS

- Loss functions can be classified into two major categories depending upon the type of learning task we are dealing with —
 1. **Regression losses**
 2. **Classification losses**
- In classification, we are trying to predict output from set of finite categorical values
- Regression, on the other hand, deals with predicting a continuous value for example given floor area, number of rooms, size of rooms, predict the price of room.

REGRESSION LOSSES

- Regression, deals with predicting a continuous value for example given floor area, number of rooms, size of rooms, predict the price of room.
- Regression losses are:
 - ✓ Mean Squared Error (L2 loss)
 - ✓ Mean Absolute Error (L1 loss)
 - ✓ Huber Loss

Mean Squared Error(L2 loss or Quadratic loss)

- *Mean square error* is measured as the average of squared difference between predictions and actual observations.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y - y_i)^2$$

n=Total number of observation

y_i = predicted value

y = actual value.

Variations of Mean Squared Error

- Half of the Mean Squared Error

$$MSE = \frac{1}{2n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

- Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2}$$

Example of MSE

Sample	Predicted	Actual	Error	Squared Error
1	48	60	-12	144
2	51	53	-2	4
3	57	60	-3	9

$$\frac{1}{3} \sum_{i=1}^3 144 + 4 + 9 = \frac{157}{3} = 52.3$$


BIG!!!
MINIMIZE

Mean Absolute Error(L1 Loss)

- *Mean absolute error(MAE)*, on the other hand, is measured as the average of sum of absolute differences between predictions and actual observations.

$$MAE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

L1 or L2 LOSS

- L2 loss is more sensitive to outliers than L1 loss
- When there are large deviations, the error is big, and when squaring a big number, it gets bigger

Example: If error is 10, then MEA would give 10 and MSE would give 100

- Consequently L1 loss function is more robust and is generally not affected by outliers
- On the other hand L2 loss function will try to adjust the model according to these outliers values even on the expense of the other samples
- Hence L2 loss function is highly sensitive to outliers in the dataset

HUBER LOSS(SMOOTH L1 LOSS)

- The Huber loss combines the best properties of MSE and MAE.
- It is quadratic for smaller errors and is linear otherwise (and similarly for its gradient).
- It is identified by its *delta* parameter:

$$\begin{aligned} \text{Huber} &= \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 & |y_i - \hat{y}_i| \leq \delta \\ \text{Huber} &= \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) & |y_i - \hat{y}_i| > \delta \end{aligned}$$

CLASSIFICATION LOSSES

- Classification losses are:
 - ✓ Binary cross entropy
 - ✓ Categorical cross entropy
 - ✓ Hinge Loss

BINARY CROSS ENTROPY LOSS

- Usually used when output labels have 0 or 1
- Also used when output labels have values between 0 and 1
- It is widely used when we have two classes (0 or 1, yes or no)

$$BCE\ Loss = -\frac{1}{n} \sum_{i=1}^n [y \log(p) + (1 - y) \log(1 - p)]$$

- This is also called **Log-Loss**.
- To calculate the probability p , we can use the sigmoid function.

Notice that when actual label is 1 ($y = 1$), second half of function disappears whereas in case actual label is 0 ($y = 0$) first half is dropped off.

CATEGORICAL CROSS ENTROPY

- Categorical Cross Entropy loss is essentially Binary Cross Entropy Loss expanded to multiple classes.
- Binary cross-entropy is for binary classification and categorical cross
- Categorical cross-entropy is based on the assumption that only 1 class is correct out of all possible ones

CATEGORICAL CROSS ENTROPY

- *The target for multi-class classification is a one-hot vector, meaning it has 1 on a single position and 0's everywhere else.*
- If our target is a one-hot vector, we can indeed forget targets and predictions for all the other classes and compute only the loss for the hot class.

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

- This is called **categorical cross-entropy** — a special case of cross-entropy, where our target is a one-hot vector.

HINGE LOSS

- Hinge loss is primarily used with Support Vector Machine (SVM) Classifiers with class labels -1 and 1
- Hinge Loss not only penalizes the wrong predictions but also the right predictions that are not confident.

$$\text{For all samples: } \sum \max(0, 1 - y * \hat{y})$$

Where y is the actual label (-1 or 1) and $\hat{y}$ is the prediction.

The loss is 0 when the signs of the label and prediction match.

$$\text{For all samples: } \sum \max(0, 1 - y * \hat{y})$$

Consider the prediction when the actual label is -1:

$\max[0, 1 - (-1 * 0.3)] = \max[0, 1.3] = 1.3 \rightarrow$ Loss is high

$\max[0, 1 - (-1 * -0.8)] = \max[0, 0.2] = 0.2 \rightarrow$ Loss is low

$\max[0, 1 - (-1 * -1.1)] = \max[0, -0.1] = 0 \rightarrow$ No Loss!

$\max[0, 1 - (-1 * -1)] = \max[0, 0] = 0 \rightarrow$ No Loss!

$\max[0, 1 - (-1 * 1)] = \max[0, 2] = 0 \rightarrow$ Loss is very high

Calculate the huber loss for the following data if $\delta=0.5$

actual values of y	4	-1.5	5	2
predicted values of $\hat{y}$	3.5	1	5	3

$$\delta = 0.5$$

$$\begin{aligned} \text{Huber} &= \frac{\frac{(4 - 3.5)^2}{2} + 0.5 \cdot \left(|-1.5 - 1| - \frac{1}{2} \cdot 0.5\right) + \frac{(5 - 5)^2}{2} + 0.5 \cdot \left(|2 - 3| - \frac{1}{2} \cdot 0.5\right)}{4} = \\ &= \frac{0.125 + 1.125 + 0 + 0.375}{4} \approx 0.406 \end{aligned}$$

$$\begin{aligned} \text{Huber} &= \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 & |y_i - \hat{y}_i| \leq \delta \\ \text{Huber} &= \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) & |y_i - \hat{y}_i| > \delta \end{aligned}$$

GRADIENT DESCENT

- Gradient descent is an optimization algorithm which is commonly-used to train machine learning models and neural networks.
- It is the base algorithm that is used to minimize the error with respect to the weights of a neural network
- Gradient descent is an optimization algorithm that works iteratively to find the model parameters with minimal cost or error values.
- Until the function is close to or equal to zero, the model will continue to adjust its parameters to yield the smallest possible error.

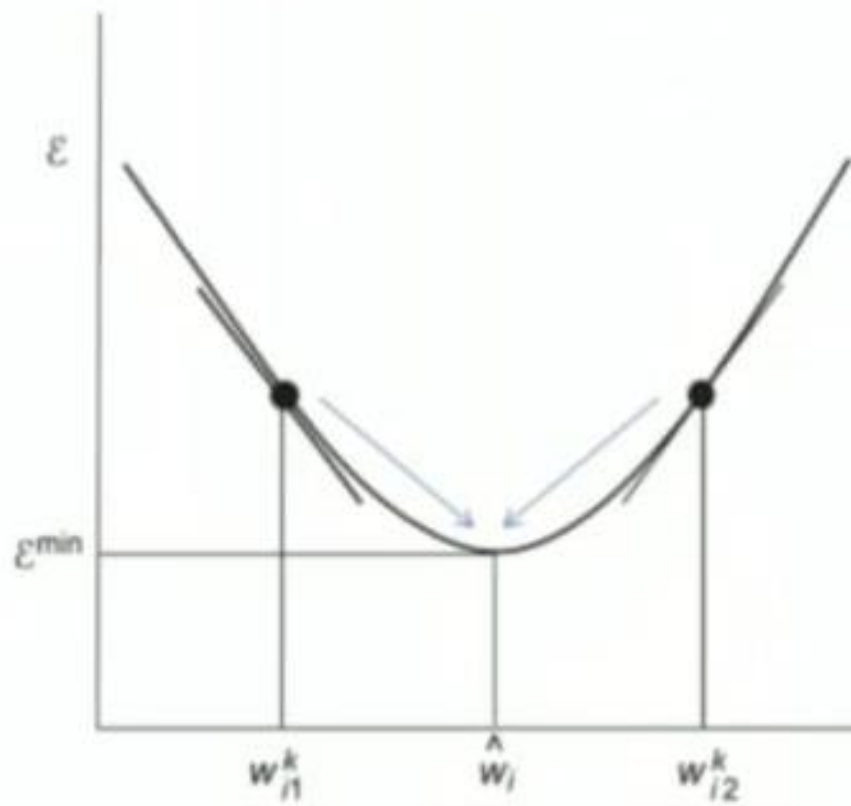
Gradient Descent

Negative Slope:

When we **increase** w , the loss function is **decreasing** $\rightarrow$
 $-(-) = + \rightarrow$ Weight Increases (Moving Right)

Positive Slope:

When we **increase** w , the loss function is **increasing**.
 $-(+) = - \rightarrow$ Weight Decreases (Moving Left)



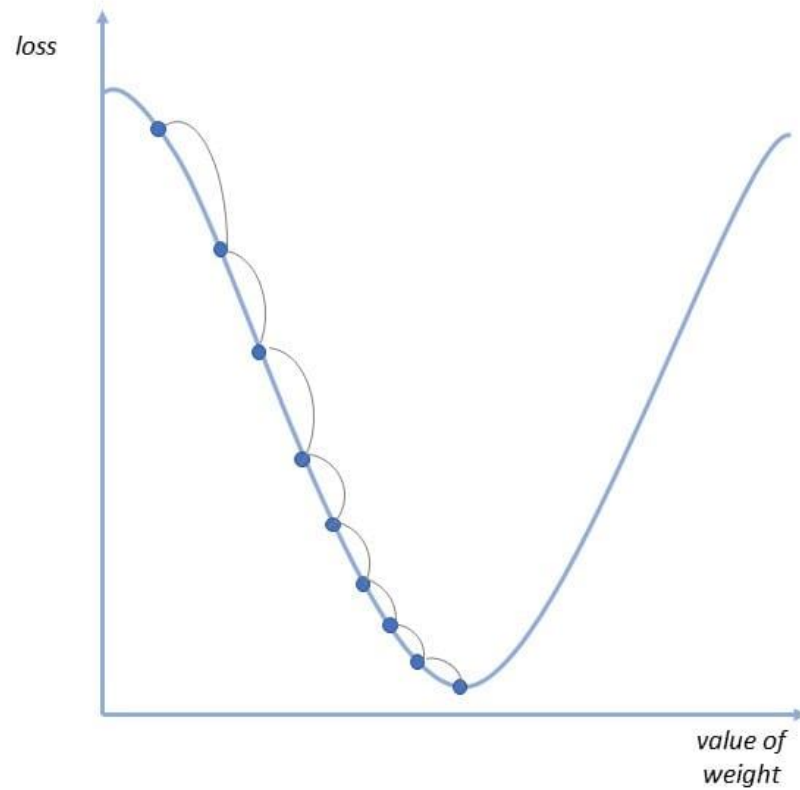
$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}$$

Learning rate(step size)

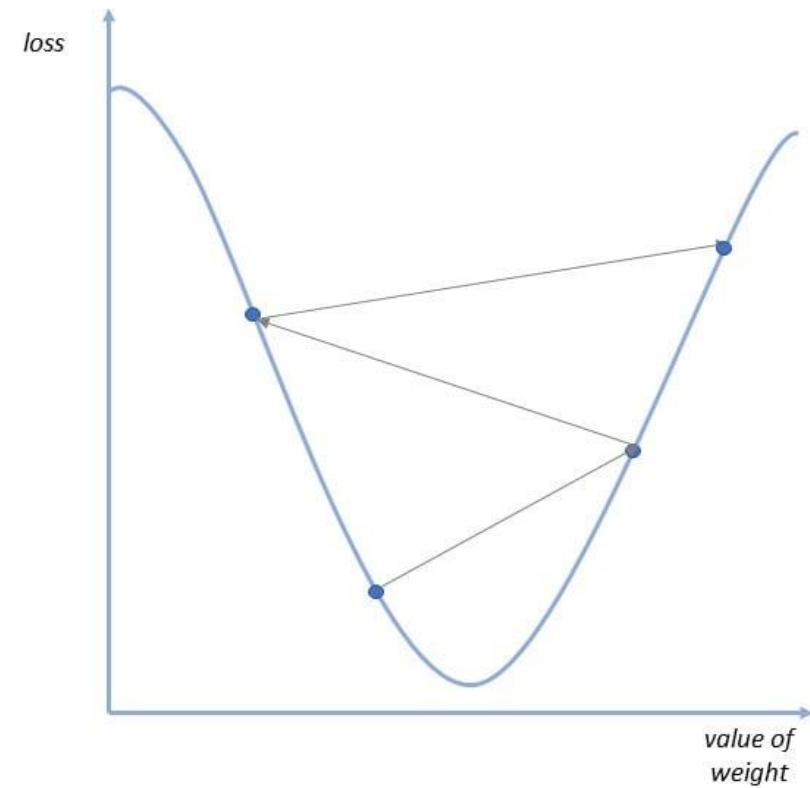
- **Learning rate** (also referred to as step size or the alpha) is the size of the steps that are taken to reach the minimum.
- This is typically a small value, and it is evaluated and updated based on the behavior of the cost function.
- High learning rates result in larger steps but risks overshooting the minimum.
- Conversely, a low learning rate has small step sizes. While it has the advantage of more precision, the number of iterations compromises overall efficiency as this takes more time and computations to reach the minimum.

Learning rate - how fast we update the weights

Small Learning Rate



Large Learning Rate



TRAINING MLP WITH BACKPROPAGATION

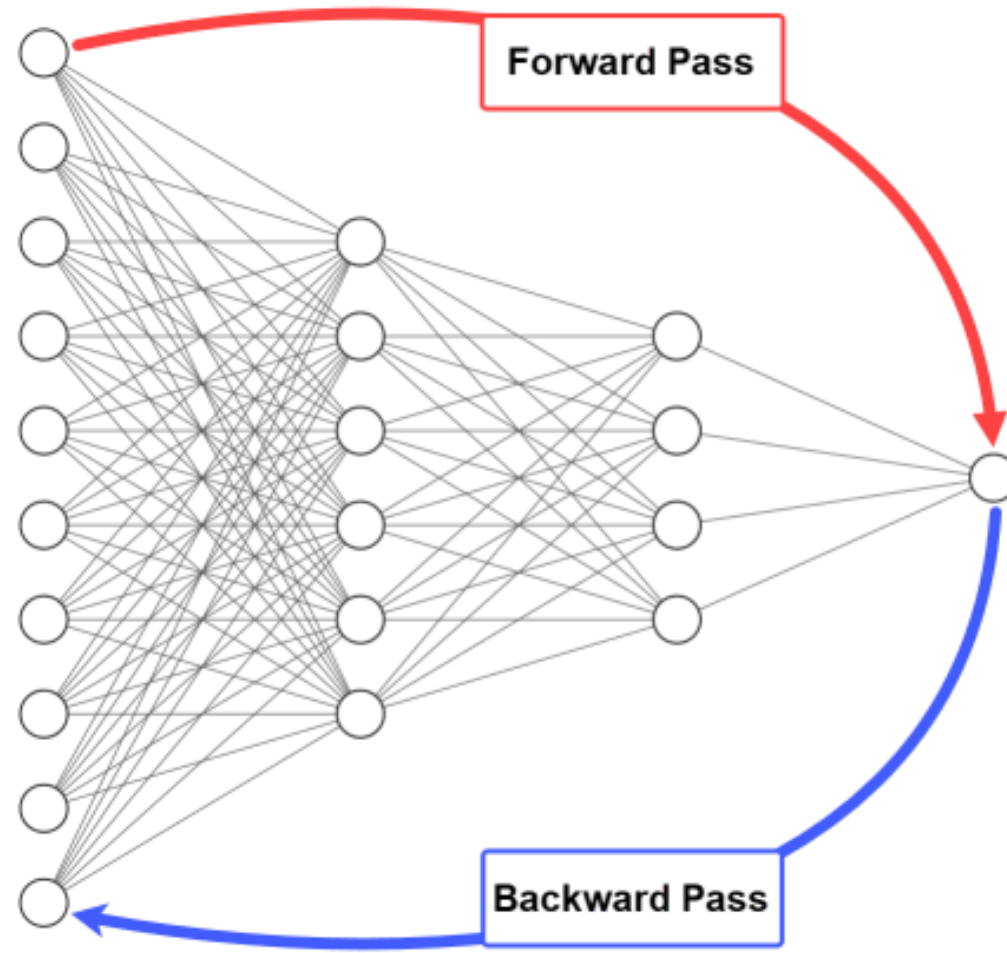
- Backpropagation algorithms are a set of methods used to efficiently train artificial neural networks following a gradient descent approach
- **Backpropagation** algorithm calculates the gradient of the **error function**.
- Backward Propagation is the preferable method of adjusting or correcting the weights to reach the minimized loss function
- It contains two main phases, referred to as the *forward* and *backward* phases, respectively

TRAINING MLP WITH BACKPROPAGATION

1. *Forward phase:* In this phase, the inputs for a training instance are fed into the neural network.
 - This results in a forward cascade of computations across the layers, using the current set of weights.
 - The final predicted output can be compared to that of the training instance and the derivative of the loss function with respect to the output is computed.
 - The derivative of this loss now needs to be computed with respect to the weights in all layers in the backwards phase.

TRAINING MLP WITH BACKPROPAGATION

- *Backward phase:* The main goal of the backward phase is to learn the gradient of the loss function with respect to the different weights by using the chain rule of differential calculus.
- These gradients are used to update the weights.
- Since these gradients are learned in the backward direction, starting from the output node, this learning process is referred to as the backward phase.



Backpropagation Algorithm

Initialize all weights to small random numbers.

Until satisfied, do

– For each training example, do

- Input the training example to the network and compute the network outputs

- For each output unit k

$$\delta_k \leftarrow o_k(1 - o_k)(y_k - o_k)$$

- For each hidden unit h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_k w_{h,k} \delta_k$$

- Update each network weight

Update the weights between hidden and output layer

$$w_{h,k} = w_{h,k} + \alpha \delta_k o_h$$

Update the weights between input and hidden layer

$$w_{x,k} = w_{x,k} + \alpha \delta_h o_x$$

o_k - predicted output

y_k - actual output

o_h - output of hidden layer

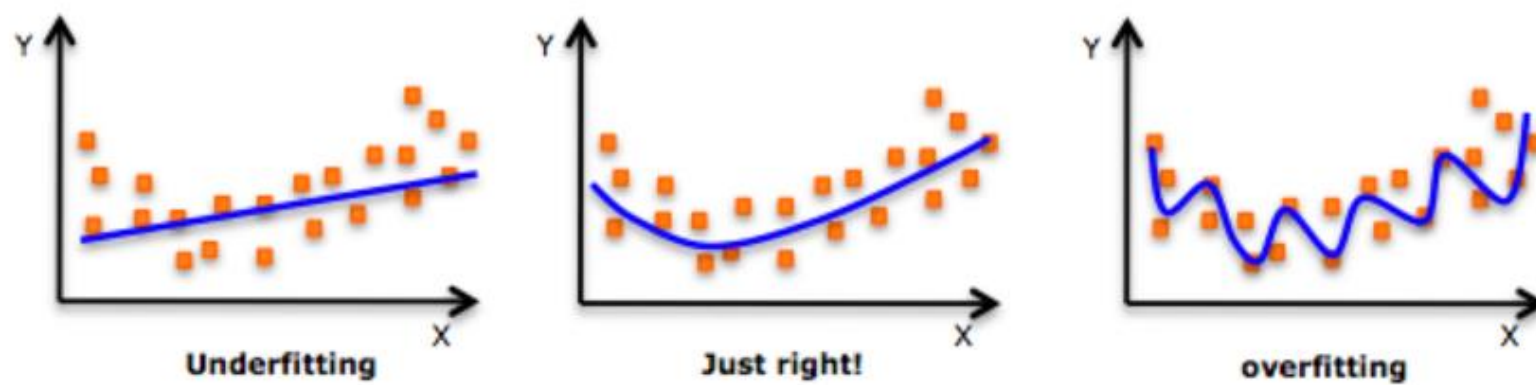
o_x - input (output of input layer)

Practical Issues in NN training

1. The problem of overfitting
2. Vanishing and exploding gradient problems
3. Difficulties in convergence
4. The problem of local optima
5. Computational challenges

The problem of overfitting

- Overfitting occurs when the model tries to make predictions on data that is very noisy.
- The problem of overfitting refers to the fact that fitting a model to a particular training data set does not guarantee that it will provide good prediction performance on unseen test data, even if the model predicts the targets on the training data perfectly.
- In other words, there is always a gap between the training and test data performance, which is particularly large when the models are complex and the data set is small



An example of overfitting, underfitting and a model that's "just right!"

Reasons for overfitting

- **The size of the training dataset is small**

When the network tries to learn from a small dataset it will tend to have greater control over the dataset & will make sure to satisfy all the data points exactly. So, the network is trying to memorize every single data point and failing to capture the general trend from the training dataset.

- **The model tries to make predictions on Noisy Data**

Overfitting also occurs when the model tries to make predictions on data that is very noisy, which is caused due to an overly complex model having too many parameters. So, due to this, the overfitted model is inaccurate as the trend does not reflect the reality present in the data.

Methods to Avoid Overfitting of a Model

- Regularization
- Train on more data
- Data Augmentation
- Early stopping
- Ensemble methods

Regularization

- Regularization refers to a set of different techniques that lower the complexity of a neural network model during training, and thus prevent the overfitting.
- Regularization techniques prevent the model from overfitting by modifying the cost function.
- In the context of machine learning, regularization is the process which regularizes or shrinks the coefficients towards zero. In simple words, regularization discourages learning a more complex or flexible model, to prevent overfitting.
- There are three very popular and efficient regularization techniques called $L1$, $L2$, and dropout

Training with more data

- One of the ways to prevent overfitting is by training with more data
- The more data we can train our model on, the more it will be able to learn from the training set.
- With more data, we're hoping to be adding more diversity to the training set as well.
- As the user feeds more training data into the model, it will be unable to overfit all the samples and will be forced to generalize to obtain results.
- Users should continually collect more data as a way of increasing the accuracy of the model.

Data augmentation

- Diversity of data and a larger dataset is the easiest way to avoid overfitting of the model.
- Data augmentation allows you to increase the size of your dataset by performing processes like flipping, cropping, rotation, scaling and translation on the existing images.

Data Augmentation



Get different versions of what you already have!
Achieve better performance without any extra labels!

Data augmentation

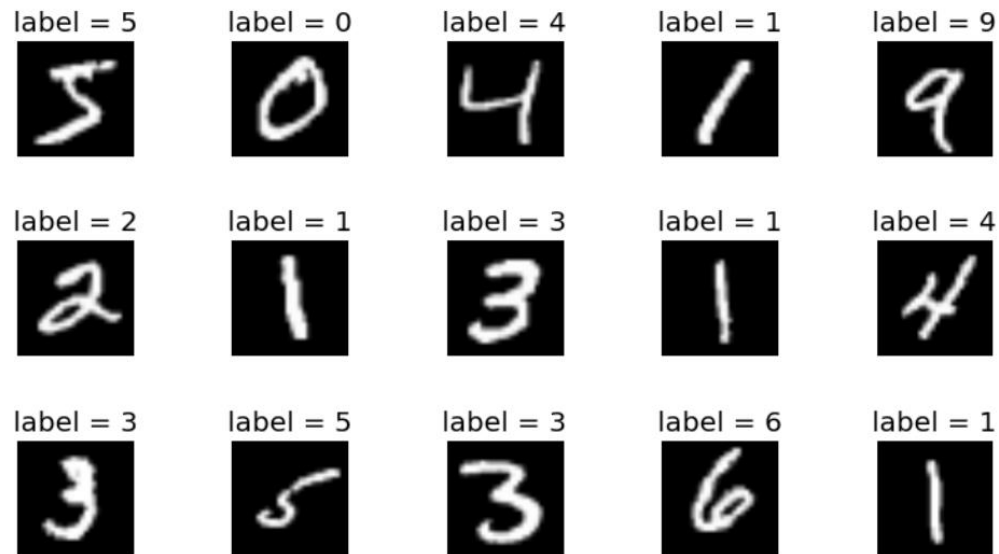
- A common trick to reduce overfitting is the idea of *data augmentation*.
- In data augmentation, new training examples are generated by using transformations on the original examples.
- Image processing is one domain to which data augmentation is very well suited.
- This is because many transformations such as translation, rotation, patch extraction, and reflection do not fundamentally change the properties of the object in an image.
- However, they do increase the generalization power of the data set when trained with the augmented data set. For example, if a data set is trained with mirror images and reflected versions of all the bananas in it, then the model is able to better recognize bananas in different orientations.

Data augmentation

- Many of the forms of data augmentation require very little computation, and therefore the augmented images do not need to be explicitly generated up front.
- Rather, they can be created at training time, when an image is being processed.
- For example, while processing an image of a banana, it can be reflected into a modified banana at training time.
- Similarly, the same banana might be represented in somewhat different color intensities in different images, and therefore it might be helpful to create representations of the same image in different color intensities
- Some forms of transformation that use principal component analysis (PCA) can be more expensive.
- PCA is used in order to change the color intensity of an image.
- If the computational costs are high, it becomes important to extract the images up front and store them.

Data augmentation

- One must be careful not to apply data augmentation blindly without regard to the dataset and application at hand.
- For example, applying rotations and reflections on the MNIST (Modified National Institute of Standards and Technology database) data set of handwritten digits is a bad idea because the digits in the data set are all presented in a similar orientation.
- Furthermore, the mirror image of an asymmetric digit is not a valid digit, and a rotation of a '6' is a '9.'



Center Crop

crops the given image at the center

Original image



Crop size=30



Crop size=50



Crop size=100



Crop size = image size

Random Resized Crop

crops an image at a random location, and then resizes the crop to a given size

Original image



Five Crop

crops the given image into four corners and the central crop

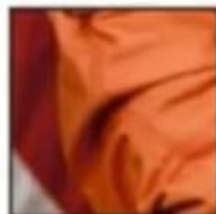
Original image



top left



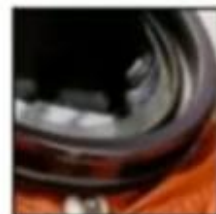
top right



bottom left



bottom right



center

Color Jitter

randomly changes the brightness, saturation, and other properties of an image

Original image



Random Rotation

rotates an image with random angle

Original image



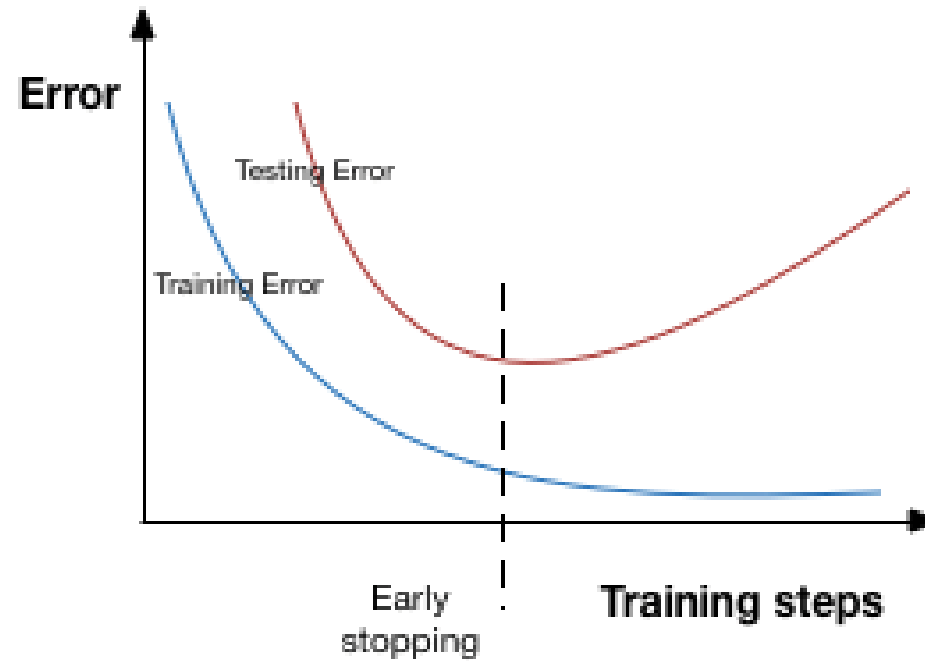
Early stopping

- Early stopping is a kind of cross-validation strategy where we keep one part of the training set as the validation set.
- When we see that the performance on the validation set is getting worse, we immediately stop the training on the model(*early stopping*, in which the gradient descent is ended after only a few iterations). This is known as early stopping.
- The main idea behind this technique is that while fitting a neural network on training data, consecutively, the model is evaluated on the unseen data or the validation set after each iteration.
- So if the performance on this validation set is decreasing or remaining the same for the certain iterations, then the process of model training is stopped.

Early stopping

- In this method, a portion of the training data is held out as a validation set.
- The backpropagation-based training is only applied to the portion of the training data that does not include the validation set.
- At the same time, the error of the model on the validation set is continuously monitored.
- At some point, this error begins to rise on the validation set, even though it continues to reduce on the training set.
- This is the point at which further training causes overfitting. Therefore, this point can be chosen for termination.

Early stopping



In the image given, we will stop training at the dotted line since after that our model will start overfitting on the training data.

The vanishing and exploding gradient problem

What is a Gradient?

- *The Gradient is nothing but a derivative of loss function with respect to the weights. It is used to update the weights to minimize the loss function during the back propagation in neural networks.*

What is Vanishing Gradients?

- *Vanishing Gradient occurs when the derivative or slope will get smaller and smaller as we go backward with every layer during backpropagation.*
- *When weight update is very small or exponential small, the training time takes too much longer, and in the worst case, this may completely stop the neural network training.*
- *A vanishing Gradient problem occurs with the sigmoid and tanh activation function because the derivatives of the sigmoid and tanh activation functions are between 0 to 0.25 and 0–1. Therefore, the updated weight values are small, and the new weight values are very similar to the old weight values. This leads to Vanishing Gradient problem. We can avoid this problem using the ReLU activation function because the gradient is 0 for negatives and zero input, and 1 for positive input.*

The vanishing and exploding gradient problem

What is Exploding Gradients?

- *Exploding gradient occurs when the derivatives or slope will get larger and larger as we go backward with every layer during backpropagation. This situation is the exact opposite of the vanishing gradients.*
- *This problem happens because of weights, not because of the activation function. Due to high weight values, the derivatives will also be higher so that the new weight varies a lot from the older weight, and the gradient will never converge. So it may result in oscillating around minima and never come to a global minima point.*

- This is primarily caused by the chain-like product computation, which can either exponentially increase or decay over the length of the path.
- In order to understand this point, consider a situation in which we have a multi-layer network with one neuron in each layer.
- Each local derivative along a path can be shown to be the product of the weight and the derivative of the activation function. The overall backpropagated derivative is the product of these values.
- If each such value is randomly distributed, and has an expected value less than 1, the product of these derivatives will drop off exponentially fast with path length.
- If the individual values on the path have expected values greater than 1, it will typically cause the gradient to explode.

Difficulties in Convergence

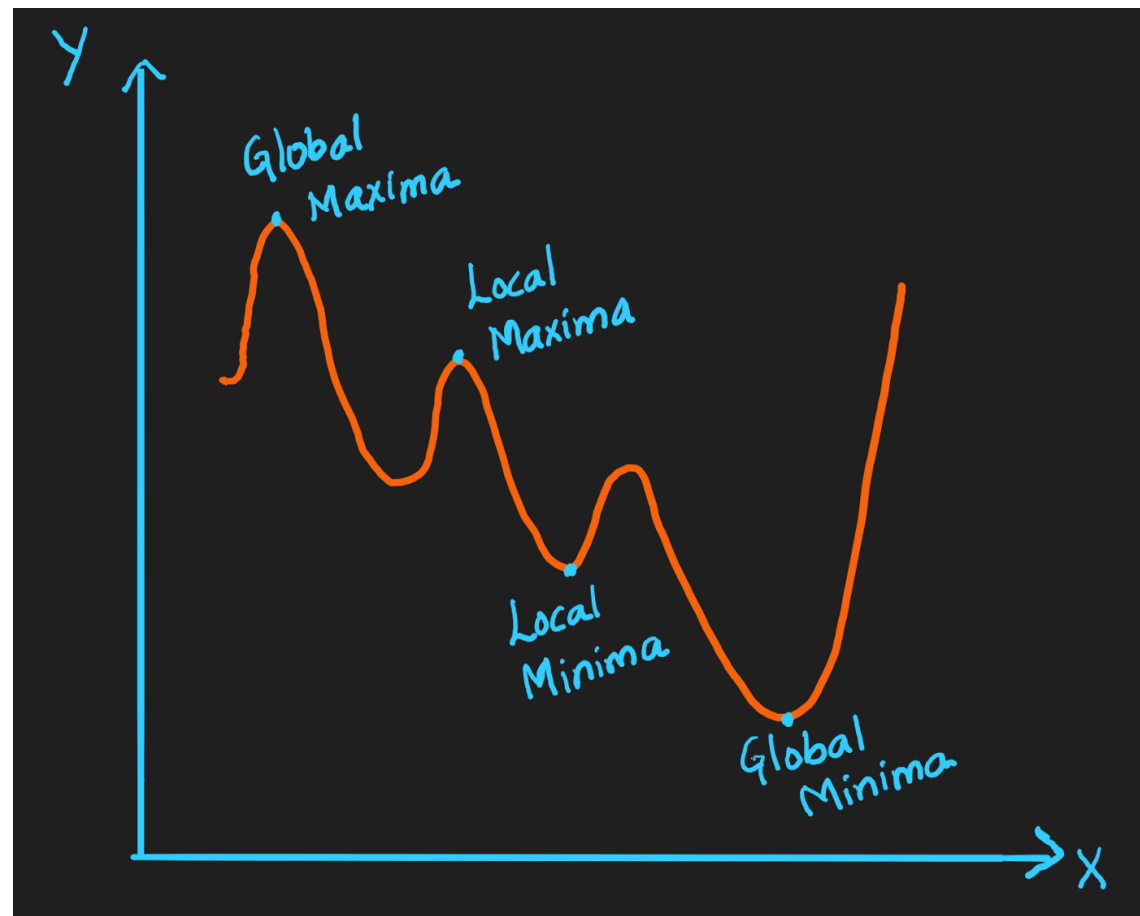
- Sufficiently fast convergence of the optimization process is difficult to achieve with very deep networks, as depth leads to increased resistance to the training process in terms of letting the gradients smoothly flow through the network.

Computational challenges

- A significant challenge in neural network design is the running time required to train the network.
- It is not uncommon to require weeks to train neural networks in the text and image domains.
- In recent years, advances in hardware technology such as Graphics Processor Units (GPUs) have helped to a significant extent.
- GPUs are specialized hardware processors that can significantly speed up the kinds of operations commonly used in neural networks.
- Although algorithmic advancements have played a role in the recent excitement around deep learning, a lot of the gains have come from the fact that the same algorithms can do much more on modern hardware.
- These modern hardware and processing units are costly and consume a lot of power.

Maxima and minima

- *Global Maxima and Minima*: It is the maximum value and minimum value respectively on the entire domain of the function
- *Local Maxima and Minima*: It is the maximum value and minimum value respectively of the function within a given range.
- There can be only one global minima and maxima but there can be more than one local minima and maxima.



The problem of local optima

- The optimization function of a neural network is highly nonlinear, which has lots of local optima
- When the parameter space is large, and there are many local optima
- In this situation, conventional training of neural networks often gets stuck in the local minima
- Another issue which could hamper the learning rate is the presence of the saddle points. Saddle points are the flat region in the error surface, which makes the gradient value nearly zero.
- Recent studies indicate that in high dimensions, saddle points are more likely than local minima.
- Saddle points are also more problematic than local minima because close to a saddle point the gradient can be very small.
- Thus, gradient descent will result in negligible updates to the network and hence network training will cease.